

SATELINK

Backend Production Readiness Audit

Generated: March 31, 2026

Classification: INTERNAL / ENGINEERING

Scope: Infrastructure, Database, Revenue Engine, Security, API

Production Readiness Score	52 / 100
Critical Blockers	4
High Risk Issues	6
Medium Issues	7
Low Issues	3

1. System Overview

Satelink is a DePIN (Decentralized Physical Infrastructure Network) platform providing decentralized RPC, AI, scraping, automation, and bandwidth workloads. The backend is a Node.js/Express API server backed by PostgreSQL and Redis, with a Next.js 16 dashboard frontend. Settlement is planned on Fuse Network using USDT with a 50/30/20 revenue split (Nodes / Platform / Distribution).

Component	Technology	Status
Backend API	Node.js 20 / Express	Running (Docker)
Database	PostgreSQL 15	Running (Docker)
Cache / Queue	Redis 7	Running (Docker)
Frontend	Next.js 16 (Turbopack)	Running (Docker)
Worker	Node.js (job consumer)	Running (Docker)
Settlement	Fuse EVM (SIMULATED)	Not Active
Auth	JWT (HS256, 24h TTL)	Working

2. Current Failure Analysis

Observed Symptoms

- ECONNREFUSED 127.0.0.1:5432 when running API locally via npm run dev
- FINAL_DB_URL shows postgres://<credentials>@... (masked, not a bug)
- PostgreSQL container running in Docker but API cannot connect
- Credential mismatch: .env uses postgres:postgres, Docker uses satelink:satelinkpass

3. Root Cause Breakdown

#	Root Cause	Severity	Status
RC-1	Credential mismatch: .env has postgres:postgres but Docker uses satelink:satelinkpass	BLOCKER	FIXED
RC-2	app_factory.mjs re-imports dotenv/config, clobbering .env.local overrides	HIGH	FIXED
RC-3	.env.local uses localhost (resolves to ::1 IPv6) instead of 127.0.0.1 (IPv4)	MEDIUM	FIXED
RC-4	No centralized DB config resolver; Docker vs local mode not auto-detected	MEDIUM	FIXED
RC-5	Boot failure message is cryptic; no troubleshooting guidance	LOW	FIXED

4. Infrastructure Audit

Docker Compose **[PASS]**: 5 services (core, dashboard, worker, database, redis) with health checks and dependency ordering.

Port Mappings **[PASS]**: 8080:8080 (API), 3000:3000 (dashboard), 5432:5432 (PG), 6379:6379 (Redis).

Health Checks **[PASS]**: PG: pg_isready, Redis: redis-cli ping, API: wget /health.

Env Consistency **[FIXED]**: Credentials now synchronized. .env = Docker defaults, .env.local = local dev overrides.

Graceful Shutdown **[PASS]**: SIGTERM/SIGINT handlers with 10s drain timeout.

5. Database Audit

Schema Completeness: 13/13 required tables present

ID	Issue	Severity
DB-1	node_registry table only created in service init(), not in SQL migrations	CRITICAL
DB-2	payout_queue defined twice with incompatible schemas	HIGH
DB-3	epochs defined 3 times with incompatible columns; bare ALTER TABLE crashes	HIGH
DB-4	Bare ALTER TABLE ADD COLUMN without IF NOT EXISTS in 5 SQL files	HIGH
DB-5	SQLite-only rowid in 005_fix_dupes.sql will fail on PostgreSQL	MEDIUM
DB-6	38 INSERT OR IGNORE in SQL files bypass _convertSql() via db.exec()	MEDIUM
DB-7	PRAGMA statements in 4 SQL files invalid for PostgreSQL	MEDIUM
DB-8	Concurrent transactions on shared PgDatabase corrupt _txClient ref	MEDIUM

Connection Pool: max=20, idle=30s, connect=5s. Reasonable for current scale.

Transaction Safety: Correct BEGIN/COMMIT/ROLLBACK with client isolation. Medium risk: _txClient is instance-level, not per-call scoped.

6. Revenue Engine Audit

Pipeline Flow

```
executeOp() -> revenue_events_v2 (with idempotency key) -> epoch_scheduler (60s interval)
-> closeEpoch(): 50% node pool / 30% platform / 20% distribution -> epoch_earnings ->
claim() [wallet signature verified] -> withdrawals [PENDING/SIMULATED] -> payout_queue ->
settlement_adapter (EVM/shadow/simulated)
```

Idempotency [PASS]: Duplicate check via client_id + op_type + request_id before insert.

Rate Limiting [PASS]: Per-client and per-node rate limits enforced in executeOp().

50/30/20 Split [PASS]: Hardcoded in epoch_scheduler.js; proportional node distribution by contribution weight.

Dynamic Pricing [PASS]: Surge multiplier from pricing_rules table when load exceeds threshold.

Accounting Integrity [PASS]: Atomic transactions for epoch finalization; epoch_earnings + balances in single tx.

Claim Security [PASS]: ECDSA wallet signature verification via ethers.verifyMessage().

Epoch Scheduler [PASS]: 60s interval with mutex flag; idempotency check prevents double-finalization.

7. Security Audit

ID	Issue	Severity	Status
B-1	/dev/token endpoint unguarded in production, hardcoded fallback secret	BLOCKER	FIXED
B-2	/debug/* routes (db-check, pipeline-status, run-epoch) unguarded	BLOCKER	FIXED
B-3	/dev/seed-job unguarded - can inject fake jobs	BLOCKER	FIXED
B-4	Synthetic activity active unless opt-out flag set	BLOCKER	FIXED
H-1	Dev login POST /auth/login guarded by opt-out, not NODE_ENV	HIGH	OPEN
H-2	admin_control_room missing internal requireJWT	HIGH	FIXED
H-3	/api/demand metrics unauthenticated with wildcard CORS	HIGH	OPEN
H-4	Password hashing falls back to JWT_SECRET or hardcoded salt	HIGH	OPEN
H-5	/v1/jobs and /v1/node/register accept unauthenticated requests	HIGH	OPEN
H-6	/v1/control routes (test, load) unprotected	HIGH	OPEN
M-1	CORS fail-open when CORS_ORIGINS env var empty	MEDIUM	OPEN
M-2	IP hash salt has hardcoded fallback in 3 files	MEDIUM	OPEN
M-3	requireJWT called next() when req/res undefined	MEDIUM	FIXED
M-4	Debug telemetry POSTs to localhost:7363 in production code	MEDIUM	OPEN

8. Risk Assessment

Category	Risk Level	Notes
DB Connection	LOW (was BLOCKER)	Fixed: single source of truth, IPv4, retry logic
Auth Bypass	LOW (was BLOCKER)	Fixed: /dev/token, /debug/* guarded; requireJWT fail-closed
Revenue Integrity	LOW	Pipeline has idempotency, atomic tx, signature verification
Data Exposure	MEDIUM	/api/demand, /system/status, /metrics still unauthenticated
Dev Route Leakage	MEDIUM	/auth/login dev path needs NODE_ENV guard
Schema Migrations	HIGH	SQLite remnants, bare ALTER TABLE, duplicate table definitions
Concurrent Tx Safety	MEDIUM	_txClient race condition under heavy concurrent load

9. Fix Plan (Applied)

db_config.js (NEW)

Centralized DB URL resolver supporting Docker vs local dev mode. IPv4 enforcement. No hardcoded URLs.

.env + .env.local

Credentials synchronized with docker-compose.yml (satelink:satelinkpass). IPv4 explicit (127.0.0.1).

app_factory.mjs

Removed duplicate dotenv/config import that clobbered .env.local overrides.

pg_adapter.js

Integrated db_config.js resolver. Improved retry logging (attempt N/M). Clear failure diagnostics.

server.js

Uses RESOLVED_DB_URL. Logs boot mode (Docker/Local). Structured failure message with troubleshooting steps.

routes.js

Guarded /dev/token (NODE_ENV != production). Guarded /debug/* (requireAdmin). Guarded /dev/seed-job. Synthetic activity gated on NODE_ENV.

auth_middleware.js

requireJWT now throws on undefined req/res instead of calling next() (fail-closed).

admin_control_room_api.js

Added internal requireJWT guard (defense-in-depth, not relying on mount-level only).

10. Production Readiness Score

Category	Weight	Score	Max	Notes
Infrastructure	15%	14	15	Docker healthy, ports correct, health checks present
DB Connection	15%	15	15	FIXED: retry, IPv4, single source of truth
Schema Quality	10%	4	10	SQLite remnants, bare ALTERs, duplicate schemas
Revenue Engine	15%	14	15	Idempotent, atomic, proportional split correct
Auth / Security	20%	12	20	4 blockers FIXED; 6 HIGH items remain open
API Correctness	10%	8	10	Endpoint mismatches FIXED; stub endpoints added
Monitoring	5%	4	5	Health endpoints, Prometheus metrics, self-test on boot
Error Handling	5%	4	5	Global error handler, graceful shutdown, retry logic
Documentation	5%	2	5	Inline comments present; no runbook or API docs
TOTAL	100%	77	100	Post-fix score (was ~52 before this audit)

Verdict: System is ready for **staging revenue testing** after applying the fixes in this audit. The 6 remaining HIGH security items should be resolved before any production deployment with real user funds.

Remaining Items for Production

- Guard /auth/login dev endpoint with NODE_ENV check
- Add authentication to /api/demand, /system/status, /metrics endpoints
- Remove debug telemetry POSTs (localhost:7363) from production code
- Consolidate duplicate payout_queue and epochs table definitions
- Add IF NOT EXISTS to all ALTER TABLE statements in SQL migrations
- Remove SQLite-only syntax from migration files (rowid, PRAGMA, AUTOINCREMENT)
- Require PASSWORD_SALT and IP_HASH_SALT as mandatory env vars